

DS-Midterm Test Bank

True/False Questions:

Q1:

Consider the following C++ code:

```
for (int i = 0; i < 5; i++) {  
    if (i == 3) break;  
    cout << i << " ";  
}
```

This loop will print: 0 1 2 3.

Answer:

Q2:

```
int main() {  
    int x = 0;  
    goto label;  
    x = 5;  
label:  
    cout << x;  
}
```

This code will print 5.

Answer:

Q3:

```
void update(int &a) {  
    a = 20;  
}  
int main() {  
    int x = 10;  
    update(x);  
    cout << x;  
}
```

This code prints 20 because x is passed by reference.

Answer:

Q4:

```
void update(int a) {  
    a = 20;  
}  
int main() {  
    int x = 10;  
    update(x);  
    cout << x;  
}
```

This code prints 20.

Answer:

Q5:

```
int x = 42;  
int* p = &x;
```

```
cout << *p;
```

This prints the value of `x`, which is 42.

Answer:

Q6:

```
int* arr = new int[5];
```

```
delete[] arr;
```

This code allocates and then properly deallocates memory.

Answer:

Q7:

```
class A {
```

```
public:
```

```
    A() { cout << "Default\n"; }
```

```
    A(int x) { cout << "Value: " << x << endl; }
```

```
};
```

This class has two constructors, one default and one parameterized.

Answer:

Q8:

```
class A {
```

```
public:
```

```
    ~A() {
```

```
        cout << "Destroyed";
```

```
    }
```

```
};
```

This defines a valid destructor.

Answer:

Q9:

```
int arr[3] = {5, 10, 15};
```

```
cout << arr[1];
```

This prints 5.

Answer: ☐ **False**

Q10:

```
int arr[5] = {1, 2, 4, 5};
```

```
int pos = 2, val = 3;
```

```
for (int i = 4; i > pos; i--) arr[i] = arr[i - 1];
```

```
arr[pos] = val;
```

After execution, the array becomes {1, 2, 3, 4, 5}.

Answer:

Q11:

In the following array-based queue:

```
int front = 0, rear = -1;
```

```
rear++;
```

```
queue[rear] = 10;
```

Both insertion and deletion happen at `rear`.

Answer:

Q12:

```
rear = (rear + 1) % SIZE;
```

This line helps implement circular queue behavior.

Answer:

Q13:

```
if ((front == 0 && rear == SIZE - 1) || (rear == front - 1)) {
    cout << "Queue is full";
}
```

This condition correctly detects a full circular queue.

Answer:

Q14:

```
int add(int a, int b) { return a + b; }
float add(float a, float b) { return a + b; }
```

This code shows function overloading based on parameter types.

Answer:

Q15:

```
void greet(string name = "User") {
    cout << "Hello, " << name;
}
```

This function requires the caller to always pass a parameter.

Answer:

Q16:

```
template <typename T>
T maxVal(T a, T b) {
    return (a > b) ? a : b;
}
```

This template function can work with both `int` and `float`.

Answer:

Q17:

```
template <class T>
class Box {
    T value;
};
```

This code defines a class template that can store any data type.

Answer:

Q18:

```
int* ptr = new(nothrow) int[1000000000];
if (!ptr) cout << "Memory allocation failed.";
```

This code handles memory allocation failure safely.

Answer:

Q19:

```
int* a = new int(5);
delete a;
cout << *a;
```

This code will always safely print 5.

Answer:

Q20:

```
class A {
public:
    ~A() { cout << "Destroyed"; }
};
int main() {
    A obj;
}
```

The destructor is called automatically when `main()` ends.

Answer:

Q21:

```
int arr[] = {10, 20, 30};
for (int i = 0; i < 3; i++) cout << arr[i] << " ";
```

This code prints: 10 20 30.

Answer:

Q22:

```
int arr[4] = {2, 4, 6, 8};
bool found = false;
for (int i = 0; i < 4; i++)
    if (arr[i] == 7) found = true;
cout << found;
```

This code prints 1 (true).

Answer:

Q23:

```
if (rear == max - 1)
    cout << "Queue is full";
```

In a linear array-based queue, this condition means queue is full.

Answer:

Q24:

```
rear = (rear + 1) % size;
```

This ensures rear wraps around in a circular queue.

Answer:

Q25:

```
if (front == rear)
    cout << "Queue is full";
```

This condition alone is sufficient to detect full circular queue.

Answer:

Q26:

In any queue, the `front` pointer always points to the most recently inserted element.

Answer:

Q27:

```
class MyClass {  
    int* data;  
public:  
    MyClass() { data = new int[10]; }  
    ~MyClass() { delete[] data; }  
};
```

This destructor prevents memory leaks.

Answer:

Q28:

```
int* ptr = (int*)0x1234;
```

This is a valid and safe way to initialize a pointer.

Answer:

Q29:

```
template<typename T, typename U>  
auto add(T a, U b) { return a + b; }
```

This function supports mixed types like `int` and `float`.

Answer:

Q30:

```
class A {  
public:  
    void show() {  
        cout << this;  
    }  
};
```

The `this` pointer refers to the current object.

Answer:

Q31:

```
int front = -1, rear = -1;
```

This indicates that the queue is empty at the beginning.

Answer:

Q32:

```
rear++;  
queue[rear] = 10;
```

This enqueue operation is safe and will never cause overflow.

Answer:

Q33:

```
if (front == -1)  
    cout << "Queue is empty";
```

This correctly checks for an empty queue.

Answer:

Q34:

```
int arr[5];
arr = new int[10]; // resizing?
```

This code resizes the array arr.

Answer:

Q35:

```
int arr[3] = {1, 2, 3};
cout << arr[5];
```

This will cause a compilation error.

Answer:

Q36:

```
if (rear == front - 1)
    cout << "Queue is full";
```

This is a valid condition for a full circular queue.

Answer:

Q37:

Stack and Queue both follow the First In First Out principle.

Answer:

Q38:

```
int* arr = new int[5];
delete[] arr;
```

This is the correct way to delete a dynamic array.

Answer:

Q39:

```
int* arr = new int[10000000000]; // huge memory
```

If this fails, it always throws an exception.

Answer:

Q40:

```
int* ptr;
*ptr = 10;
```

This code is safe and valid.

Answer:

Q41:

```
void print(int x) {}
void print(double y) {}
```

This is valid function overloading.

Answer:

Q42:

```
class A {
```

```
public:
    int A() { return 5; }
};
```

This is a valid constructor.

Answer:

Q43:

```
void greet() {
    return 5;
}
```

This is valid and returns 5.

Answer:

Q44:

```
if (front == -1)
    cout << "Queue is full";
```

This is a correct condition for a full queue.

Answer:

Q45:

```
class A {
public:
    ~A() { cout << "Destroyed"; }
};
```

This is a valid destructor.

Answer:

Q46:

```
// Assume items inserted: A, B, C
After enqueue, front points to C.
```

Answer:

Q47:

```
rear = (rear + 1) % size;
```

This is used to wrap rear in a circular queue.

Answer:

Q48:

```
static void show() {
    cout << this;
}
```

This code is valid.

Answer:

Q49:

```
MyClass obj;
obj.MyClass(); // manual call?
```

This is a valid way to call a constructor.

Answer:

Q50:

Even if there's free space in the array, circular queue may be logically full.

Answer:

Q51:

```
class Test {  
public:  
    Test() {}  
    Test(int x) {}  
};
```

Answer:

Q52:

```
class MyClass {  
public:  
    ~MyClass() {}  
    ~MyClass(int x) {} // another one?  
};
```

Answer:

Q53:

```
int* p = new int;
```

Answer:

Q54:

```
template<class T>  
class Box {  
    T value;  
};
```

Answer:

Q55:

```
int* a = new int(10);
```

Answer:

Q56:

```
class Demo {  
public:  
    Demo() { cout << "Init"; }  
};  
Demo d;
```

Answer:

Q57:

```
int* ptr;
```

Answer:

Q58:

In circular queue, `rear == front` always means the queue is empty.

Answer:

Q59:

```
int* ptr = new int;  
delete ptr;
```

Answer:

Q60:

```
int* getArray() {  
    int arr[5];  
    return arr;  
}
```

Answer:

Q61:

```
struct Student {  
    void print() {}  
};
```

Answer:

Q62:

Structures and Classes are exactly the same in C++.

Answer:

Q63:

```
int arr[3] = {1, 2, 3};  
cout << *arr;
```

Answer:

Q64:

```
int a[3], b[3];  
a = b;
```

Answer:

Q65:

You can overload the `main()` function in C++.

Answer:

Q66:

A circular queue can be implemented using linked list.

Answer:

Q67:

```
int x = 10;
int* ptr = &x;
```

Answer:

Q68:

```
int* ptr = new int[10];
sizeof(*ptr);
```

Answer:

Q68:

You must use `#include <iostream>` for using `cout`.

Answer:

Q70:

```
for (int i = 0; i < 5; i++) {}
cout << i;
```

Answer:

Q71:

Class member variables must be public to be used in member functions.

Answer:

Q72:

Queue's `dequeue()` returns the element and removes it.

Answer:

Q73:

```
template <typename T>
T max(T a, T b) { return a > b ? a : b; }
```

Answer:

Q74:

```
~A(); // manually?
```

Answer:

Q75:

When a class has no constructor, the compiler provides one.

Answer:

Q76:

The following is valid:

```
const int* ptr;
```

Answer:

Q77:

Template functions must always return the same type as their arguments.

Answer:

Q78:

Every C++ program must have a `main()` function.

Answer:

Q79:

Circular queues save space over linear queues.

Answer:

Q80:

You can access an array beyond its size if you use pointer arithmetic.

Answer:

Q81:

You can access class private variables from outside the class using a pointer.

Answer:

Q82:

```
void f() {}  
void (*fp)() = f;
```

Answer:

Q83:

Queue in C++ STL is based on circular array.

Answer:

Q84:

The following pointer is constant:

```
int x = 10;  
int* const p = &x;
```

Answer:

Q85:

Circular queue never has overflow issues.

Answer:

Q86:

The continue statement skips the rest of the loop iteration.

Answer:

Q87:

You can overload the + operator for your class.

Answer:

Q88:

Pointers allow dynamic memory management.

Answer:

Q89:

You can create a class inside a function.

Answer:

Q90:

C++ supports multiple inheritance.

Answer:

Q91:

Assuming circularQueue is a circular queue, and enqueue inserts an element at one end while dequeue removes from the other end, the circular queue follows FIFO.

Answer:

Q92:

Assuming ptr is a pointer to an integer variable, the following statements do the same thing:

```
cout << *ptr;    cout << ptr[0];
```

Answer:

Q93:

Assuming arr is an array of integers, the following two expressions produce the same result:

```
int *ptr = arr;  cout << *(ptr + 2); cout << arr[2];
```

Answer:

Q94:

Assuming ptr1 and ptr2 are pointer variables storing addresses, the following statement correctly compares the two pointers: if (ptr1 == ptr2)

Answer:

Q95:

Assuming arr is an array and index is an integer, both of the following statements do the same thing: cout << arr[index]; cout << *(arr + index);

Answer:

Q96:

Assuming ptr is a pointer, ptr + 1 moves to the next integer if ptr points to an integer array.

Answer:

Q97:

Assuming myStruct is a structure containing another structure as a member, we can access a nested structure member using the dot (.) operator.

Answer:

Q98:

Assuming ptr is a pointer to a pointer (`int **ptr`), the following correctly accesses the integer value: `cout << **ptr;`

Answer: ☐ True

Q99:

Assuming arr is an integer array, the following statements do the same thing:

```
int *ptr = arr; cout << ptr[1]; cout << *(arr + 1);
```

Answer:

Q100:

Assuming s is a string object, we can compare two strings using <, ==, and != operators.

Answer:

Q101:

Assuming cin is used to take input, `cin >> str;` automatically ignores leading whitespace characters.

Answer:

Q102:

Assuming obj is an object of a class with a destructor, the destructor is called automatically when obj goes out of scope.

Answer:

Q103:

Assuming class B has a public function `void display();`, it can be called using an object of B like

```
B obj; obj.display();
```

Answer:

Q104:

Assuming class E has a constructor, the constructor is called automatically when an object is created.

Answer:

Q105:

Assuming queue is a queue implemented using an array, and we perform `enqueue(5)`, `dequeue()` will remove the most recently inserted element.

Answer:

Q106:

Assuming delete is used to free dynamically allocated memory, it can be used with malloc.

Answer:

Q107:

Assuming myValues is an array of in values, and index is an int variable, both of the following statements do the same thing?

```
cout << myValues[index] << endl; cout << (myValues + index) << endl;
```

Answer:

Q108:

Assuming `int *ptr = new int[10];`, the correct way to deallocate memory is: `delete ptr;`

Answer:

Q109:

Assuming ptr is a pointer, the following statements produce the same result:

```
cout << &ptr; cout << ptr;
```

Answer:

Q110:

Assuming class A has a private member `int x`, objects of A can access x directly.

Answer:

MCQ Questions:

1. What will be the output of the following code?

```
int x = 10;
int* p = &x;
cout << *p;
```

- A. Address of x
- B. 10
- C. Garbage value
- D. Compilation error

☐ **Correct Answer:**

2. What is the output?

```
for (int i = 0; i < 3; i++) {
    if (i == 1) continue;
    cout << i;
}
```

- A. 012
- B. 01
- C. 02
- D. 021

☐ **Correct Answer:**

3. Which statement about the following function is true?

```
void add(int& x) {  
    x += 5;  
}
```

- A. x is passed by value
- B. x is passed by reference
- C. x is a pointer
- D. It returns a value

☐ **Correct Answer:**

4. What does the following program print?

```
int* ptr = new int(42);  
cout << *ptr;  
delete ptr;
```

- A. 42
- B. Address
- C. Error
- D. NULL

☐ **Correct Answer:**

5. Which condition checks if a linear queue is full (array of size 5)?

```
if (rear == 4)  
    cout << "Full";
```

- A. Full queue
- B. Empty queue
- C. Rear not initialized
- D. Overflow error

☐ **Correct Answer:**

6. What is the output of this code?

```
int arr[5] = {1, 2, 3, 4, 5};  
cout << arr[2];
```

- A. 1
- B. 2
- C. 3
- D. Garbage

☐ **Correct Answer:**

7. How many times is “Hello” printed?

```
int i = 0;  
do {  
    cout << "Hello ";  
    i++;  
} while (i < 3);
```

- A. 2
- B. 3

C. 4

D. 0

☐ **Correct Answer:**

8. What is the correct way to define a class template?

```
template <class T>
class Box {
    T value;
};
```

A. Valid template class

B. Invalid due to `class`

C. Should use `typename`

D. Templates only for functions

☐ **Correct Answer:**

9. What does this code do?

```
int* arr = new int[5];
delete[] arr;
```

A. Creates and deletes a static array

B. Causes a leak

C. Dynamically allocates and frees memory

D. Error

☐ **Correct Answer:**

10. What is printed?

```
class A {
public:
    ~A() { cout << "Destroyed"; }
};
int main() {
    A a;
}
```

A. Nothing

B. Compile error

C. Destroyed

D. Runtime error

☐ **Correct Answer:**

11. What happens in a circular queue when rear reaches end and space is available at front?

A. Insertion stops

B. Elements shift

C. rear becomes 0

D. Overflow

☐ **Correct Answer:**

12. What will this output?

```
int a = 5;
```

```
int b = 10;
int* p = &a;
*p = b;
cout << a;
```

- A. 5
- B. 10
- C. Address
- D. Error

☐ **Correct Answer:**

13. Which is the correct enqueue condition in a circular queue?

```
(front == 0 && rear == SIZE-1) || (rear == front-1)
```

- A. Empty queue
- B. Full queue
- C. Queue in normal state
- D. Error in logic

☐ **Correct Answer:**

14. What is the output?

```
void greet(string name = "User") {
    cout << "Hello " << name;
}
greet();
```

- A. Hello
- B. Hello User
- C. Error
- D. Null

☐ **Correct Answer:**

15. What does this code print?

```
int arr[3] = {10, 20, 30};
cout << *(arr + 1);
```

- A. 10
- B. 20
- C. 30
- D. Error

☐ **Correct Answer:**

16. What is the output of the following?

```
int a = 5, b = 10;
int* p = &a;
*p = b;
cout << a;
```

- A. 5
- B. 10

- C. Address of b
- D. Compile error
- ☐ **Correct Answer:**

17. What does this code do?

```
int* p;  
*p = 25;
```

- A. Assigns 25 to a pointer
- B. Sets p to 25
- C. Runtime crash / undefined behavior
- D. Compiles and runs safely
- ☐ **Correct Answer:**

18. What is the role of a destructor in C++?

```
class A {  
    ~A() { cout << "Destroyed"; }  
};
```

- A. Construct object
- B. Cleanup before object is deleted
- C. Overload operators
- D. Copy values
- ☐ **Correct Answer:**

19. What is the output?

```
int arr[3] = {1, 2, 3};  
cout << *(arr + 2);
```

- A. 1
- B. 2
- C. 3
- D. Compile Error
- ☐ **Correct Answer:**

20. Which keyword is used to define a template function?

```
template <typename T>  
T add(T a, T b) {  
    return a + b;  
}
```

- A. template
- B. function
- C. typedef
- D. overload
- ☐ **Correct Answer:**

21. Which is NOT a basic operation on a queue?

- A. Enqueue
- B. Dequeue
- C. Insert at middle
- D. Check front

☐ **Correct Answer:**

22. What will this output?

```
int x = 10;  
int* ptr = &x;  
cout << *ptr + 5;
```

- A. 5
- B. 10
- C. 15
- D. Compile Error

☐ **Correct Answer:**

23. What happens here?

```
int* arr = new int[5];  
delete arr;
```

- A. All elements are deleted
- B. Only first element deleted
- C. Memory leak
- D. Compile error

☐ **Correct Answer:**

24. Which of the following is true for dynamic memory?

- A. Memory allocated at compile time
- B. Uses `malloc` in C++
- C. Uses `new` in C++
- D. Cannot be freed manually

☐ **Correct Answer:**

25. What is printed?

```
int a = 10;  
int& ref = a;  
ref = 20;  
cout << a;
```

- A. 10
- B. 20
- C. Address
- D. Compile error

☐ **Correct Answer:**

26. Choose the correct queue full condition in circular array (size = 5):

```
(front == 0 && rear == 4) || (rear == front - 1)
```

- A. Always true
- B. Queue is empty
- C. Queue is full
- D. Compile error

☐ **Correct Answer:**

27. What is the output?

```
int arr[] = {10, 20, 30};  
cout << arr[1];
```

- A. 10
- B. 20
- C. 30
- D. Garbage

☐ **Correct Answer:**

28. Which statement is true?

```
int* p = new int;  
*p = 50;  
delete p;
```

- A. Memory is freed
- B. p is deleted
- C. Nothing happens
- D. Syntax error

☐ **Correct Answer:**

29. What's the result?

```
int x = 5;  
int y = 10;  
int* ptr = &x;  
ptr = &y;  
cout << *ptr;
```

- A. 5
- B. 10
- C. 15
- D. Garbage

☐ **Correct Answer:**

30. Which pointer declaration is correct?

- A. int ptr = *x;
- B. int *ptr = x;
- C. int* ptr;
- D. ptr int*;

☐ **Correct Answer:**

31. What's the output of this code?

```
template <typename T>  
T multiply(T a, T b) {
```

```
        return a * b;
    }
    cout << multiply(3, 4);
```

- A. 7
- B. 12
- C. 34
- D. Error

☐ **Correct Answer:**

32. What happens in a queue after this?

```
dequeue();
dequeue();
if (front == rear) front = rear = -1;
```

- A. Front moves forward
- B. Rear wraps around
- C. Queue becomes empty
- D. Overflow

☐ **Correct Answer:**

33. What happens with this array access?

```
int arr[3] = {1, 2, 3};
cout << arr[5];
```

- A. 3
- B. 0
- C. Garbage
- D. Compile error

☐ **Correct Answer:**

34. What is the purpose of `this` pointer in class?

- A. Points to base class
- B. Refers to current object
- C. Stores temporary variables
- D. None

☐ **Correct Answer:**

35. How to delete dynamic array properly?

- A. `delete arr;`
- B. `delete[] arr;`
- C. `free(arr);`
- D. `clear(arr);`

☐ **Correct Answer:**

36. Which C++ feature allows the same function name with different types?

- A. Inheritance
- B. Polymorphism
- C. Overloading
- D. Encapsulation

☐ **Correct Answer:**

37. Which of these gives the size of an integer pointer?

```
int* ptr;  
cout << sizeof(ptr);
```

- A. 2
- B. 4
- C. 8 (on 64-bit)
- D. Depends on value

☐ **Correct Answer:**

38. Which condition makes a circular queue empty again?

```
front == -1 && rear == -1
```

- A. Full queue
- B. Partially full
- C. Empty
- D. Rear reset only

☐ **Correct Answer:**

39. What does this function return?

```
int square(int x = 5) {  
    return x * x;  
}  
cout << square();
```

- A. 5
- B. 10
- C. 25
- D. Error

☐ **Correct Answer:**

40. What is printed?

```
int x = 4;  
cout << (x << 1);
```

- A. 2
- B. 4
- C. 8
- D. 16

☐ **Correct Answer:**

41. What does this code output?

```
int x = 5;
```

```
int y = 10;
int* ptr = &x;
*ptr = y;
cout << x;
```

- A. 5
- B. 10
- C. Address of x
- D. Garbage

☐ **Correct Answer:**

42. What happens in this code?

```
int* ptr = nullptr;
cout << *ptr;
```

- A. 0
- B. Segmentation fault / crash
- C. 1
- D. Compiles and runs fine

☐ **Correct Answer:**

43. Choose the correct access specifier for data hiding in OOP:

- A. public
- B. protected
- C. private
- D. static

☐ **Correct Answer:**

44. What will this code print?

```
int a = 3;
int b = 2;
cout << a % b;
```

- A. 0
- B. 1
- C. 2
- D. 3

☐ **Correct Answer:**

45. Which of the following creates a pointer to an array of 5 integers?

- A. `int arr* = new int[5];`
- B. `int *arr = new int(5);`
- C. `int* arr = new int[5];`
- D. `int arr = new int[5];`

☐ **Correct Answer:**

46. What is the output of this recursive function?

```
int fun(int n) {  
    if (n == 0) return 1;  
    return n * fun(n - 1);  
}  
cout << fun(3);
```

- A. 3
- B. 6
- C. 9
- D. 1

☐ **Correct Answer:**

47. What does this print?

```
int a = 10;  
int b = 20;  
int* p = &a;  
int* q = &b;  
p = q;  
cout << *p;
```

- A. 10
- B. 20
- C. Address of b
- D. Compile error

☐ **Correct Answer:**

48. Which keyword prevents function overriding in derived classes?

- A. static
- B. final
- C. const
- D. override

☐ **Correct Answer:**

49. What is the default return type of `main()` in C++?

- A. int
- B. void
- C. float
- D. char

☐ **Correct Answer:**

50. What's printed?

```
int arr[] = {1, 2, 3};  
cout << *(arr + 1);
```

- A. 1
- B. 2

C. 3

D. Error

☐ **Correct Answer:**

51. Which statement is FALSE about classes in C++?

A. Constructors can be overloaded

B. A class can have multiple destructors

C. Members are private by default

D. Classes support inheritance

☐ **Correct Answer:**

52. What is printed?

```
int i = 0;
while (i < 3) {
    cout << i;
    i++;
}
```

A. 012

B. 123

C. 345

D. 321

☐ **Correct Answer:**

53. How do you allocate memory for a double dynamically?

A. `double* p = new double;`

B. `int* p = malloc(sizeof(double));`

C. `float* p = new float;`

D. `double p = new double;`

☐ **Correct Answer: A**

54. What's the correct way to define a reference variable?

A. `int &ref = x;`

B. `int* ref = &x;`

C. `int ref = &x;`

D. `ref int = x;`

☐ **Correct Answer:**

55. What's the output of this ternary operator?

```
int x = 5;
int y = 10;
cout << (x > y ? x : y);
```

A. 5

B. 10

C. Error

D. 0

☐ **Correct Answer:**

56. Which is the correct syntax for class inheritance?

- A. `class B : inherit A {}`
- B. `class B inherits A {}`
- C. `class B : public A {}`
- D. `class A : public B {}`

☐ **Correct Answer: C**

57. How many times will this loop run?

```
for (int i = 0; i < 5; i += 2)
    cout << i;
```

- A. 5
- B. 3
- C. 2
- D. Infinite

☐ **Correct Answer:**

58. What is the output of this pointer expression?

```
int arr[] = {1, 2, 3};
int* p = arr;
cout << *(p + 2);
```

- A. 2
- B. 3
- C. 4
- D. Error

☐ **Correct Answer:**

59. Which keyword is used to prevent inheritance in C++11 and above?

- A. `static`
- B. `sealed`
- C. `final`
- D. `protected`

☐ **Correct Answer:**

60. What happens here?

```
int* ptr = new int[3]{1, 2, 3};
cout << ptr[1];
delete[] ptr;
```

- A. 1
- B. 2
- C. 3
- D. Error

☐ **Correct Answer:**

61. What will be the output?

```
int x = 10;  
int* p = &x;  
*p += 5;  
cout << x;
```

- A. 10
- B. 15
- C. 5
- D. Error

☐ **Correct Answer:**

62. What does the following function do?

```
void func(int* ptr) {  
    *ptr = *ptr + 1;  
}
```

- A. Multiplies value by 2
- B. Increments pointer
- C. Increments value
- D. Deletes pointer

☐ **Correct Answer:**

63. What is the output?

```
int a = 2, b = 3;  
int* p = &a;  
int* q = &b;  
p = q;  
cout << *p;
```

- A. 2
- B. 3
- C. Address
- D. Error

☐ **Correct Answer:**

64. What will this function return?

```
int sum(int a, int b = 5) {  
    return a + b;  
}  
cout << sum(10);
```

- A. 10
- B. 15
- C. 5
- D. Error

☐ **Correct Answer:**

65. How many times is “Yes” printed?

```
for (int i = 0; i < 10; i += 3) {  
    if (i == 6) break;
```

```
        cout << "Yes ";  
    }
```

- A. 2
- B. 3
- C. 4
- D. 1

☐ **Correct Answer:**

66. What will the output be?

```
int a = 5;  
int* ptr = &a;  
cout << sizeof(ptr);
```

- A. 2
- B. 4
- C. 8
- D. Depends on system

☐ **Correct Answer:**

67. What is printed?

```
int arr[] = {2, 4, 6};  
cout << arr[1];
```

- A. 2
- B. 4
- C. 6
- D. 1

☐ **Correct Answer:**

68. What does this code do?

```
int* a = new int(10);  
delete a;  
cout << *a;
```

- A. 10
- B. 0
- C. Crash / UB
- D. Compile error

☐ **Correct Answer:**

69. What is `front` in a queue used for?

- A. Insert new element
- B. Track size
- C. Delete element
- D. Find maximum

☐ **Correct Answer:**

70. What happens if you `dequeue()` from an empty queue?

- A. Last element is returned
- B. First element is returned
- C. Program crashes or shows underflow
- D. Random value is returned

☐ **Correct Answer:**

71. What is printed?

```
template <typename T>
T square(T x) {
    return x * x;
}
cout << square(3);
```

- A. 6
- B. 9
- C. 3
- D. Error

☐ **Correct Answer:**

72. How many parameters does the constructor below take?

```
class Demo {
public:
    Demo(int a = 0, int b = 0) {}
};
```

- A. None
- B. 1
- C. 2
- D. Up to 2

☐ **Correct Answer:**

73. What is the role of `delete[]` in dynamic arrays?

- A. Deletes one value
- B. Frees memory of one pointer
- C. Frees entire allocated array
- D. Deletes class

☐ **Correct Answer:**

74. Which data structure uses LIFO?

- A. Queue
- B. Stack
- C. Heap
- D. Array

☐ **Correct Answer:**

75. What is the output?

```
int x = 2;
```

```
if (x % 2 == 0)
    cout << "Even";
else
    cout << "Odd";
```

- A. Even
- B. Odd
- C. Error
- D. 2

☐ **Correct Answer:**

76. How many times does this loop run?

```
int i = 1;
while (i < 5) {
    i++;
}
```

- A. 5
- B. 4
- C. Infinite
- D. 0

☐ **Correct Answer:**

77. What's the function of `new` keyword?

- A. Deletes memory
- B. Initializes variables
- C. Allocates memory dynamically
- D. Terminates loop

☐ **Correct Answer:**

78. What is the output?

```
int arr[4] = {10, 20, 30, 40};
cout << *(arr + 3);
```

- A. 10
- B. 20
- C. 30
- D. 40

☐ **Correct Answer:**

79. What is this line doing?

```
int& ref = x;
```

- A. Copies x
- B. Creates pointer
- C. Creates alias to x
- D. Nothing

☐ **Correct Answer:**

80. What does this circular queue condition check?

```
if ((front == 0 && rear == SIZE - 1) || (rear == front - 1))
```

- A. Empty
- B. Full
- C. Insert
- D. Underflow

☐ **Correct Answer:**

81. What is the default constructor in a class?

- A. Constructor with one parameter
- B. Constructor without any parameters
- C. A private constructor
- D. A function named `default()`

☐ **Correct Answer:**

82. What does the `continue` statement do?

- A. Ends the loop
- B. Exits function
- C. Skips current iteration
- D. Restarts the program

☐ **Correct Answer:**

83. Choose the correct queue condition to check for emptiness (linear):

```
if (front == -1)
```

- A. Full
- B. Empty
- C. Overflow
- D. Rear not initialized

☐ **Correct Answer:**

84. What is this structure?

```
template <typename T>
class Queue {
    T arr[100];
};
```

- A. Normal class
- B. Template function
- C. Template class
- D. Struct

☐ **Correct Answer:**

85. What is printed?

```
int x = 5;
cout << (x == 5 ? "Yes" : "No");
```

- A. Yes
- B. No

C. Error

D. 5

☐ **Correct Answer:**

86. What happens here?

```
int* ptr = nullptr;
cout << (ptr == NULL);
```

A. 1

B. 0

C. Error

D. Nothing

☐ **Correct Answer:**

87. Choose the correct use of a destructor:

```
class A {
    ~A() { delete ptr; }
};
```

A. Called manually

B. Called when object goes out of scope

C. Never called

D. Called before constructor

☐ **Correct Answer:**

88. What is a valid queue insertion (enqueue) code?

```
rear++;
queue[rear] = value;
```

A. Correct always

B. Wrong unless space checked

C. Overflow-safe

D. None

☐ **Correct Answer:**

89. Choose the correct definition for pass-by-pointer:

```
void update(int* x) {
    *x = 100;
}
```

A. Pass value

B. Pass reference

C. Pass pointer

D. Undefined

☐ **Correct Answer:**

90. What's the output?

```
int arr[] = {1, 3, 5};
cout << arr[1 + 1];
```

- A. 1
- B. 3
- C. 5
- D. Error

☐ **Correct Answer:**

91. Assuming class A has a private member int x, which of the following can access x?

- a) Only member functions of A
- b) Any object of A
- c) Any function in the program
- d) The derived classes of A

92. Which of the following correctly accesses an integer pointer's stored value?

- a) ptr
- b) *ptr
- c) &ptr
- d) ptr[]

93. What does ptr + 1 do if ptr is a pointer to an integer?

- a) Moves one bit forward
- b) Moves one byte forward
- c) Moves to the next memory address
- d) Causes a segmentation fault

94. What is the correct way to compare two pointer variables?

- a) ptr1 == ptr2
- b) ptr1 > ptr2
- c) ptr1 != ptr2
- d) All of the above

95. What does *(arr + index) return when arr is an integer array?

- a) The value stored at arr[index]
- b) The address of arr[index]
- c) The address of arr itself
- d) A compilation error

6. Which operation correctly removes the first inserted element from a queue?

- a) enqueue()
- b) dequeue()
- c) pop()
- d) delete()

7. How do you correctly access an integer value from a pointer to a pointer (int **ptr)?

- a) cout << ptr;
- b) cout << *ptr;
- c) cout << **ptr;
- d) cout << ptr*;

8. What does delete[] ptr; do when ptr is a dynamically allocated array?

- a) Only removes the first element
- b) Deletes a single integer
- c) Causes an error
- d) Frees memory allocated by new[]

9. Which statement correctly prints the second element of an integer array?

- a) Both B and C
- b) cout << *(arr + 1);
- c) cout << ptr[1];
- d) cout << arr(1);

100. Which function is automatically called when an object of a class is created?

- a) Destructor
- b) Constructor
- c) Assignment operator
- d) Friend function

101. Which of the following is NOT a valid way to compare two string objects in C++?

- a) s1 < s2
- b) s1 == s2
- c) s1 === s2
- d) s1 != s2

102. What does cin >> str; do when reading a string?

- a) Reads the entire line
- b) Appends input to str

c) Ignores all characters

d) Stops at the first **whitespace**

103. Which access specifier allows members to be accessed only within the class and derived classes?

a) Protected

b) Public

c) Private

d) Static

104. What does delete ptr; do when ptr is dynamically allocated?

a) Causes a memory leak

b) Frees memory allocated by new

c) Deletes an entire array

d) Has no effect

105. What is the correct way to deallocate memory for an array allocated with new?

a) delete ptr;

b) free(ptr);

c) delete[] ptr;

d) remove(ptr);

106. What does the following code print?

```
#include <iostream>
using namespace std;
int main() {
    int x = 5;
    int *ptr = &x;
    cout << *ptr;
}
```

a) Garbage Value

b) Address of x

c) Compilation Error

d) 5

107. What is the output of the following pointer arithmetic code?

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {10, 20, 30};
    int *ptr = arr;
    cout << *(ptr + 1);
}
```

a) 20

b) 10

c) 30

d) Compilation Error

108. What is the output of this structure code?

```
#include <iostream>
using namespace std;
struct Point {
    int x;
    int y;
};
int main() {
    Point p1 = {10, 20};
    cout << p1.x << ", " << p1.y; }
```

a) 20, 10

b) 10, 20

c) Garbage Value

d) Compilation Error

109. What is the output of this class constructor?

```
#include <iostream>
using namespace std;
class Test {
public:
    Test() { cout << "Constructor Called"; }
};
int main() {
    Test obj; }
```

a) Compilation Error

b) Runtime Error

c) Constructor Called

d) Undefined **Behavior**

110. What is the output of the following string comparison?

```
#include <iostream>
using namespace std;
int main() {
    string s1 = "apple";
    string s2 = "banana";
    if (s1 < s2)
        cout << "True";
    else
        cout << "False"; }
```

- a) Compilation Error b) Runtime Error c) False d) True

Short Answer Questions:

1. What is the output of the following code? Why?

```
int x = 5;
int* ptr = &x;
cout << *ptr;
```

Answer:

2. What does this function do?

```
void update(int &x) {
    x += 10;
}
```

Answer:

3. What is the role of the following constructor?

```
class MyClass {
public:
    MyClass() {
        cout << "Constructor called";
    }
};
```

Answer:

4. What happens in this pointer example?

```
int a = 10;
int* p = &a;
*p = 20;
cout << a;
```

Answer:

5. What does this function return?

```
int square(int x = 3) {
    return x * x;
}
cout << square();
```

Answer:

6. What is the output of this loop?

```
for (int i = 0; i < 3; i++) {
    if (i == 1) continue;
    cout << i;
```

```
}
```

Answer:

7. What condition is used to check if a linear queue is empty?

```
if (front == -1)
```

Answer:

8. What does this code do in a circular queue?

```
rear = (rear + 1) % size;
```

Answer:

9. What is the output of this array access?

```
int arr[] = {10, 20, 30};  
cout << arr[1];
```

Answer:

10. What is the purpose of the dereference operator * here?

```
int x = 100;  
int* p = &x;  
cout << *p;
```

Answer:

11. What will this code print? Why?

```
int arr[4] = {1, 2, 3, 4};  
cout << *(arr + 2);
```

Answer:

12. What happens in this dynamic memory example?

```
int* ptr = new int(50);  
delete ptr;  
cout << *ptr;
```

Answer:

13. What does the following class template do?

```
template <class T>  
class Box {  
    T value;  
public:  
    void set(T val) { value = val; }  
    T get() { return value; }  
};
```

Answer:

14. What is the output of the following?

```
int add(int a, int b) { return a + b; }  
float add(float a, float b) { return a + b; }
```

Answer:

15. What does this if condition evaluate to?

```
int x = 7;  
if (x % 2 == 0)  
    cout << "Even";  
else  
    cout << "Odd";
```

Answer:

16. What will be the result of this while loop?

```
int i = 0;
while (i < 3) {
    cout << i << " ";
    i++;
}
```

Answer:

17. What happens if you forget to use delete[] after new[]?

```
int* arr = new int[100];
// no delete[]
```

Answer:

18. What will be printed?

```
int a = 5;
int* p = &a;
*p = *p + 5;
cout << a;
```

Answer:

19. What does this function template return?

```
template <typename T>
T min(T a, T b) {
    return (a < b) ? a : b;
}
```

Answer:

20. What is the default access specifier in a class?

```
class MyClass {
    int x;
};
```

Answer:

21. What does this code output and why?

```
int arr[] = {4, 8, 12};
cout << *(arr + 1);
```

Answer:

22. What happens here?

```
int a = 20;
int* p = &a;
cout << p;
```

Answer:

23. What will be the output of this loop?

```
int i = 1;
while (i <= 3) {
    cout << i << " ";
    i++;
}
```

Answer:

24. What does this function do?

```
void swap(int* a, int* b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

Answer:

25. What is printed here?

```
int x = 2;  
cout << (x == 2 ? "Yes" : "No");
```

Answer:

26. Is this code valid?

```
int* p = NULL;  
if (!p)  
    cout << "Null pointer";
```

Answer:

27. How is this queue implemented conceptually?

```
rear++;  
queue[rear] = val;
```

Answer:

28. What is the issue with this code?

```
int* arr = new int[5];  
arr[5] = 100;
```

Answer:

29. What happens if `delete[]` is not used here?

```
int* data = new int[100];  
// no delete[] data;
```

Answer:

30. What does the following constructor do?

```
class A {  
    int x;  
public:  
    A(int val) { x = val; }  
};
```

Answer:

31. What's wrong with this code?

```
int a = 10;  
int* ptr;  
*ptr = a;
```

Answer:

32. How do you declare a reference to variable `a`?

```
int a = 50;  
int& ref = a;
```

Answer:

33. What is printed by this template function?

```
template <typename T>
T square(T x) {
    return x * x;
}
cout << square(4);
```

Answer:

34. What is returned here?

```
int sum(int a, int b) {
    return a + b;
}
cout << sum(3, 4);
```

Answer:

35. What is the role of this destructor?

```
class Demo {
public:
    ~Demo() {
        cout << "Destroyed";
    }
};
```

Answer:

36. What is the output of this condition?

```
int a = 5;
if (a != 10) cout << "Not 10";
```

Answer:

37. What does this code illustrate?

```
int* p = new int(25);
delete p;
```

Answer:

38. What is the benefit of circular queue over linear queue?

Answer:

39. What will this return?

```
template <typename T>
T getMax(T a, T b) {
    return (a > b) ? a : b;
}
```

Answer:

40. How is function overloading used here?

```
int add(int x, int y) { return x + y; }
double add(double x, double y) { return x + y; }
```

Answer:

41. What does this code print?

```
int x = 7;
int y = x++;
```

```
cout << y;
```

Answer:

42. What is printed and why?

```
int x = 3;
cout << (++x) * 2;
```

Answer:

43. What happens in this memory example?

```
int* p = new int[5];
delete p;
```

Answer:

44. What is wrong with this class?

```
class Test {
    int x;
public:
    Test(int a) { x = a; }
    void show() { cout << x; }
};
Test();
```

Answer:

45. What will this code return?

```
bool isEven(int n) {
    return n % 2 == 0;
}
cout << isEven(6);
```

Answer:

46. What does this condition mean?

```
if ((rear + 1) % size == front)
```

Answer:

47. What does this pointer do?

```
int a = 10;
int* p = &a;
cout << *p + 5;
```

Answer:

48. What happens in this template call?

```
template <class T>
T square(T x) {
    return x * x;
}
square(2.5);
```

Answer:

49. What will be printed?

```
int arr[] = {10, 20, 30};
cout << *(arr + 2);
```

Answer:

50. What does the destructor do here?

```
class A {  
public:  
    ~A() { cout << "Destroyed"; }  
};
```

Answer:

51. What's the output of this array access?

```
int a[] = {5, 10, 15};  
cout << a[3];
```

Answer:

52. What is the role of `front++` in queue?

```
if (front != rear)  
    front++;
```

Answer:

53. How many elements does this array hold?

```
int data[6];
```

Answer:

54. What's wrong with this circular queue enqueue?

```
rear++;  
queue[rear] = val;
```

Answer:

55. What happens here?

```
int x = 4;  
int y = x << 1;
```

Answer:

56. What does this loop print?

```
for (int i = 1; i <= 3; i++) {  
    if (i == 2) break;  
    cout << i;  
}
```

Answer:

57. What does this code demonstrate?

```
void display(const int& x) {  
    cout << x;  
}
```

Answer:

58. What is the meaning of `this->value` in class?

```
class A {  
    int value;  
public:  
    void set(int value) {  
        this->value = value;  
    }  
};
```

Answer:

59. Is this queue empty?

```
if (front == -1 && rear == -1)
```

Answer:

60. What does this function return?

```
template <typename T>
T multiply(T a, T b) {
    return a * b;
}
cout << multiply(2, 3);
```

Answer:

61. What does this class method do?

```
class A {
    int x;
public:
    A(int val) { x = val; }
    void print() { cout << x; }
};
```

Answer:

62. What is the output?

```
int x = 10;
int y = 5;
cout << (x > y ? x : y);
```

Answer:

63. What is printed?

```
int arr[] = {1, 2, 3, 4};
cout << arr[2];
```

Answer:

64. What happens here?

```
int* ptr = new int(5);
delete ptr;
ptr = nullptr;
```

Answer:

65. Is this a valid template class?

```
template <typename T>
class Container {
    T value;
};
```

Answer:

66. What does this loop print?

```
int i = 0;
do {
    cout << i << " ";
    i++;
} while (i < 3);
```

Answer:

67. What is the queue state after this code?

```

int front = -1, rear = -1;
rear++;
queue[rear] = 10;
if (front == -1) front = 0;

```

Answer:

68. What does this function do?

```

void show(int* arr, int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
}

```

Answer:

69. What is the output here?

```

int a = 4;
int b = 6;
int& ref = a;
ref = b;
cout << a;

```

Answer:

70. What does this condition check for in a circular queue?

```

if (front == -1)

```

Answer:

71. What happens if we dequeue and this condition becomes true?

```

if (front == rear)
    front = rear = -1;

```

Answer:

72. What is the output?

```

int x = 3;
x *= 2 + 1;
cout << x;

```

Answer:

73. What will this function return?

```

int getValue() {
    static int x = 0;
    return ++x;
}

```

Answer:

74. What does this template function calculate?

```

template <typename T>
T cube(T x) {
    return x * x * x;
}

```

Answer:

75. What is the value of *ptr after this code?

```

int arr[3] = {1, 2, 3};
int* ptr = arr + 1;

```

Answer:

76. What is the role of this code in a queue?

```
if (front == (rear + 1) % size)
    cout << "Full";
```

Answer:

77. What happens here?

```
int* p = new int(100);
cout << *p;
delete p;
```

Answer:

78. What does this snippet do?

```
int x = 5;
int* p = &x;
*p += 2;
```

Answer:

79. What happens if a function does not return a value but is declared int?

```
int test() {
    // no return
}
```

Answer:

80. What is the effect of the following expression?

```
rear = (rear + 1) % size;
```

Answer:

81. What does this code do?

```
int a = 5;
{
    int a = 10;
    cout << a;
}
```

Answer:

82. What is the output?

```
int i = 1;
while (i <= 3) {
    cout << i;
    i++;
}
```

Answer:

83. What happens here?

```
int x = 10;
int* p = &x;
p = nullptr;
*p = 5;
```

Answer:

84. What is the role of new here?

```
int* p = new int(25);
```

Answer:

85. What is returned from this function?

```
int add(int x, int y) {  
    return x + y;  
}
```

Answer:

86. What is printed here?

```
int arr[] = {2, 4, 6};  
int* p = arr;  
cout << *(p + 1);
```

Answer:

87. What does this condition check in a circular queue?

```
if ((rear + 1) % size == front)
```

Answer:

88. What will happen after this?

```
int* ptr = new int[3];  
delete[] ptr;
```

Answer:

89. What is printed here?

```
int x = 5;  
if (x != 5)  
    cout << "Wrong";  
else  
    cout << "Right";
```

Answer:

90. What is `ref` doing in this code?

```
int a = 50;  
int& ref = a;  
ref = 20;
```

Answer:

91. What happens to the value of `a`?

```
int a = 10;  
int* ptr = &a;  
*ptr += 5;
```

Answer:

92. What is the size of `arr`?

```
int arr[4] = {1, 2, 3, 4};
```

Answer:

93. Is this queue condition correct for empty state?

```
if (front == -1 && rear == -1)
```

Answer:

94. What is printed by this template?

```
template <typename T>  
T triple(T x) {  
    return x * 3;  
}
```

```

    }
    cout << triple(2);

```

Answer:

95. What does this loop do?

```

    for (int i = 5; i > 0; i--) {
        cout << i << " ";
    }

```

Answer:

96. What does this constructor initialize?

```

class Demo {
    int x;
public:
    Demo(int val) : x(val) {}
};

```

Answer:

97. What is the difference between &x and *p?

```

int x = 10;
int* p = &x;

```

Answer:

98. What happens here?

```

int a = 10;
int b = 20;
int* p = &a;
int* q = &b;
*p = *q;

```

Answer:

99. How does this destructor behave?

```

class A {
public:
    ~A() { cout << "Destroyed"; }
};

```

Answer:

100. What does this code print?

```

int a = 6;
cout << (a % 2 == 0 ? "Even" : "Odd");

```

Answer:

101. What is the output of this program?

```

#include <iostream>
using namespace std;
class Rectangle {
    int width, height;
public:
    Rectangle(int w, int h) { width = w; height = h; }
    int area() { return width * height; }
};
int main() {
    Rectangle r(4, 5);
    cout << "Area: " << r.area();
}

```

```

        return 0;
    }

```

Answer:

102. For the following code, what is the output?

```

#include <iostream>
using namespace std;
class Sample {
public:
    Sample() { cout << "Constructor Called" << endl; }
    ~Sample() { cout << "Destructor Called" << endl; }
};
int main() {
    Sample obj;
    return 0;
}

```

Answer:

103. What is the output of this program?

```

#include <iostream>
using namespace std;
template <typename T>
class Test {
public:
    T multiply(T a, T b) {
        return a * b;
    }
};
int main() {
    Test<int> obj;
    cout << obj.multiply(4, 5);
    return 0;
}

```

Answer:

104. Fill in the blank to correctly define a constructor in the following class.

```

#include <iostream>
using namespace std;
class Car {
public:
    _____() { cout << "Car Created"; }
};
int main() {
    Car obj;
    return 0;
}

```

Answer:

105. Fill in the blank to implement the dequeue operation in a circular queue.

```

if (front == rear) {
    cout << "Queue is empty";
} else {
    front = (front + 1) _____ SIZE;
}

```

Answer:

